

TP Vue.js / Node.js – Partie 1

Séance totale de 3h • Partie 1 : conception et réalisation initiale • Durée conseillée : 1h

1. Énoncé

Dans cette première partie, chaque binôme doit concevoir la base d'une application web de gestion de projets BTP avec une interface Kanban simple. L'objectif est de produire une première version fonctionnelle, crédible et bien structurée, permettant de suivre l'avancement de tâches liées à un chantier.

L'application doit permettre d'afficher plusieurs projets de construction et leurs tâches associées, réparties dans des colonnes de type Kanban comme : À faire, En cours, En attente, Terminé. Cette première version doit déjà inclure une interface claire, une logique métier cohérente, une structure frontend/backend propre et un code suffisamment lisible pour être repris plus tard.

Les binômes peuvent utiliser des agents IA pour accélérer le développement, mais uniquement de manière maîtrisée : le code généré doit être compris, testé, adapté et correctement commenté.

2. Contexte

Vous travaillez pour une entreprise fictive du secteur du BTP qui souhaite mieux suivre ses chantiers. Jusqu'à présent, les tâches sont mal centralisées et le suivi de l'avancement manque de clarté. L'entreprise veut donc un outil simple pour visualiser rapidement l'état des projets, les tâches en cours, les blocages et les étapes terminées.

L'application doit répondre à un besoin métier réaliste, avec une interface claire et une organisation logique des informations, adaptée au suivi de chantier.

3. Mission

Votre mission est de réaliser en 1 heure une première base exploitable de cette application.

Vous devez :

- mettre en place la structure du projet
- créer une interface Kanban simple et lisible
- afficher des projets et des tâches de chantier cohérents
- prévoir une base de données ou un jeu de données crédible
- utiliser GitHub pour versionner le travail
- commenter le code de manière claire
- produire un projet suffisamment propre pour pouvoir être repris par un autre binôme dans la suite du TP

À la fin de cette partie, le projet doit être exécutable, compréhensible, documenté et poussé sur le dépôt Git.

4. Cahier des charges fonctionnel minimal

- Une page d'accueil présentant l'application, son objectif et des liens de navigation clairs.
- Une page "Projets / Chantiers" listant les chantiers en cours sous forme de cartes.
- Une page "Tableau Kanban" pour un chantier avec 4 colonnes minimum : À faire, En cours, Bloqué, Terminé.
- Des cartes de tâches très simples : titre, chantier, responsable, échéance, priorité, statut.
- Une page détail d'une tâche ou d'un chantier accessible via Vue Router.
- Un formulaire d'ajout de tâche avec v-model et validation minimale.
- Un filtre basique par statut, priorité ou texte recherché.

- Au moins deux profils dans les données : chef de projet et conducteur de travaux.
- Un backend Express minimal avec au moins des routes GET et POST pour les projets ou les tâches.
- Une interface visuellement propre : navigation visible, colonnes lisibles, cartes homogènes, couleurs sobres inspirées du BTP (gris, bleu, orange, blanc).

5. Structure attendue

- Frontend Vue : composants réutilisables (Navbar, ProjectCard, KanbanColumn, TaskCard, TaskForm).
- Routing minimal : /, /projects, /kanban/:id, /tasks/:id.
- Backend Node/Express séparé du frontend et renvoyant du JSON.
- README obligatoire : lancement, arborescence, fonctionnalités terminées, limites connues, prochaines étapes.
- Code très bien commenté : composants, fonctions clés, appels API, logique de filtrage, structure des données.

6. Usage autorisé des agents IA

Vous pouvez utiliser l'IA pour accélérer la production de la base, mais vous devez garder une logique d'ingénierie.

- Toujours préciser le rôle demandé à l'IA, le contexte du projet, la stack, les fichiers concernés et le format de sortie attendu.
- Demander des fichiers complets, cohérents avec votre arborescence, plutôt que des réponses vagues.
- Tester immédiatement tout code généré avant de l'intégrer.
- Ne jamais copier sans relire ; l'IA peut introduire des incohérences entre frontend, backend et données.
- Documenter dans le README ce qui a été généré ou corrigé avec l'IA.

7. Exemples de prompts utiles

- Tu es un développeur Vue 3 senior. Crée un composant KanbanColumn réutilisable pour une application BuildFlow de gestion de chantiers BTP. Props : title, tasks. Contraintes : code simple, commenté, sans TypeScript, compatible Vue Router. Sortie attendue : fichier .vue complet.
- Tu es un développeur Express. Crée une API minimale pour une application BuildFlow avec les routes GET /projects, GET /tasks, POST /tasks. Contraintes : données JSON simples, code commenté, pas de base SQL pour cette première version.
- Tu es un relecteur technique. Analyse mon README et indique ce qui manque pour qu'un autre binôme puisse reprendre le projet sans aide orale. Réponse attendue : checklist courte.

8. GitHub et suivi du dépôt

Vous devez créer un dépôt GitHub pour le projet, travailler proprement et maintenir le dépôt à jour pendant toute la séance.

Action	Commande	Quand l'utiliser
Cloner un dépôt	git clone url-du-repo	Récupérer le projet
Créer une branche	git checkout -b nom-branche	Démarrer le travail du binôme
Vérifier l'état	git status	Contrôler les fichiers modifiés
Préparer un commit	git add .	Ajouter les fichiers
Créer un commit	git commit -m "message clair"	Enregistrer une étape cohérente
Publier la branche	git push origin nom-branche	Envoyer le travail sur GitHub
Mettre à jour	git pull origin nom-branche	Récupérer les dernières modifications

9. Livrables de fin de partie 1

- Une première version exécutable de l'application.
- Un dépôt GitHub initialisé, poussé et à jour.
- Un code lisible et commenté.
- Un README suffisamment clair pour permettre une reprise autonome.
- Au moins un commit intermédiaire et un commit de fin de partie.